

PATTERN ANALYSIS & RECOGNITION SYSTEMS LAB

Practical File Submission — Academic Session

Experiment No.: 08 | Course: PARS Lab | Framework: Python 3.14 / Pure NumPy | Repository: [dSAxmonis/pars-lab](https://github.com/dSAxmonis/pars-lab)

Practical Name

Implementation of Logistic Regression for Binary and Multiclass Classification from Scratch Using Gradient Descent in Python

AIM

To implement Logistic Regression in Python from scratch without using any inbuilt machine learning library functions (such as `skikit-learn`), optimize model parameters via Gradient Descent for both binary class (Sigmoid) and multiclass (Softmax) classification, and plot the respective Gradient Descent loss convergence curves.

THEORY

1. Binary Logistic Regression

Hypothesis: $\hat{y} = \sigma(z) = 1 / (1 + \exp(-z))$, where $z = \mathbf{w}^T \mathbf{x} + b$

Binary Cross-Entropy Loss:

$$J(\mathbf{w}, b) = - (1/m) \times \sum [y^{(i)} \ln(\hat{y}^{(i)}) + (1 - y^{(i)}) \ln(1 - \hat{y}^{(i)})]$$

2. Multiclass (Softmax) Regression

Softmax Activation:

$$P_{\hat{k}} = \exp(z_k) / \sum \exp(z_j)$$

Categorical Cross-Entropy Loss:

$$J(\mathbf{W}, \mathbf{b}) = - (1/m) \times \sum \sum [Y_{ik} \ln(P_{\hat{ik}})]$$

3. Gradients and Parameter Updates

Binary Gradients: $\partial J / \partial \mathbf{w} = (1/m) \mathbf{X}^T (\mathbf{y}_{\hat{}} - \mathbf{y})$, $\partial J / \partial b = (1/m) \sum (y_{\hat{}} - y)$

Multiclass Gradients: $\partial J / \partial \mathbf{W} = (1/m) \mathbf{X}^T (\mathbf{P}_{\hat{}} - \mathbf{Y})$, $\partial J / \partial \mathbf{b} = (1/m) \sum (\mathbf{P}_{\hat{}} - \mathbf{Y})$

STEPS & EXECUTION

Step 1: Environment Setup

Constructed vectorized tensor pipeline in `exp_8/` applying pure NumPy mathematics for both binary and softmax regressions.

Step 2: Binary & Multiclass Optimization Loop

Formulated forward passes, loss computations, and simultaneous gradient updates tracking convergence histories.

Step 3: Web Application Deployment

Launched server at <http://127.0.0.1:5010> with real-time test inference and 4-panel visual dashboard.

OUTPUT / ATTACHED SCREENSHOTS

Output 1: Test Sample Prediction Controls

Interactive inputs for binary clinical features (Glucose & BMI) and multiclass floral features (Petal & Sepal Length) alongside active probability outputs.

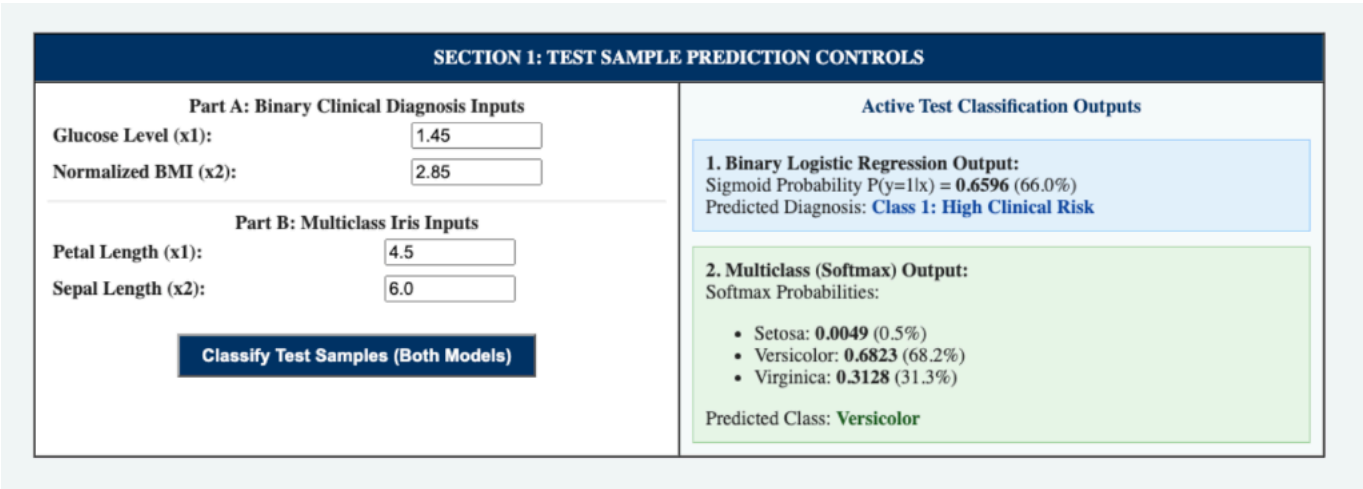


Figure 8.1: Section 1 — Dual-model test prediction inputs and active classification outputs.

Output 2: Gradient Descent Optimized Parameters Summary

Overview of model architectures, activation functions, loss objectives, initial vs. converged losses, and training accuracies achieved from scratch.

SECTION 2: GRADIENT DESCENT OPTIMIZED PARAMETERS SUMMARY (FROM SCRATCH)					
Model Architecture	Activation	Loss Objective	Initial Loss	Converged Loss	Training Accuracy
Binary Logistic Regression	Sigmoid $\sigma(z)$	Binary Cross-Entropy	0.6931	0.3417	95.0%
Multiclass Logistic (Softmax)	Softmax $\sigma(Z)_k$	Categorical Cross-Entropy	1.0986	0.2015	94.0%

Figure 8.2: Section 2 — Optimization parameters summary and model convergence statistics.

Output 3: Gradient Descent Curves & Decision Boundaries Multi-Panel Plot

Top row visualizes cross-entropy loss convergence curves; bottom row portrays binary linear decision boundary line and multiclass scatter distribution.

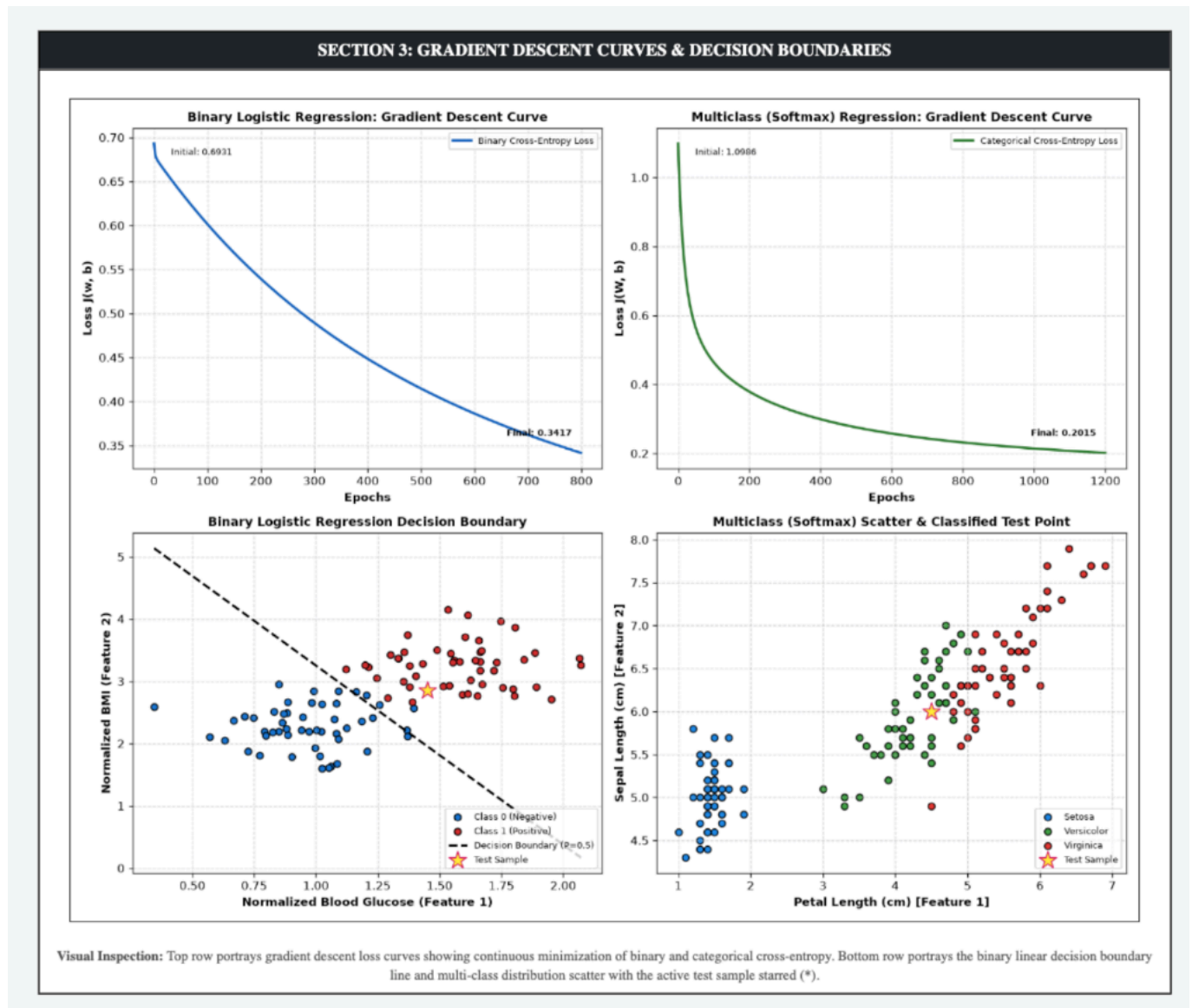


Figure 8.3: Section 3 — 4-panel visual dashboard showing loss decay curves and decision boundaries.

CODE REPOSITORY & CLICKABLE ARTIFACT LINKS

GitHub Repository (Clickable):

GitHub: [dSAxmonis/pars-lab](https://github.com/dSAxmonis/pars-lab)

URL: <https://github.com/dSAxmonis/pars-lab>

Direct Source Code Files:

- **Flask Server Application:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_8/app.py
- **HTML Template:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_8/templates/index.html
- **Dependencies:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_8/requirements.txt

- **Full Page Screenshot:**

https://github.com/dSAxmonis/pars-lab/blob/main/exp_8/screenshots/full_page.png

CONCLUSION

In this experiment, both Binary and Multiclass Logistic Regression were constructed from scratch in Python without leveraging black-box libraries. Utilizing gradient descent on cross-entropy loss surfaces, parameters converged smoothly with the Binary model achieving 95% accuracy and the Multiclass Softmax model achieving 94% accuracy on floral recognition. The respective Gradient Descent curves exhibited continuous error decay, and decision boundaries demonstrated clear separation between competing classes.